M365 & Azure Incident Response & Escalation Runbook — Government (GCC High)

Implementation Solution Guide

Version 1.0.0 · 2026-07-01

Contents

Executive Summary — M365 & Azure Incident Response & Escalation Runbook (Government / GCC High edition) 5

The business problem . . . 5

What this package delivers . . . 5

The phase model . . . 6

Outcomes . . . 6

Honest scope . . . 6

Investment . . . 6

1. Solution Overview — M365 & Azure Incident Response & Escalation Runbook (Government / GCC High edition) 6

What this solution is . . . 6

Commercial vs GCC High — what's different . . . 7

The phase model . . . 9

The named artifacts . . . 9

The per-workload diagnostic playbooks . . . 10

How it works (the flow) . . . 10

Benefits . . . 11

Honest limitations . . . 11

The standard runbook model . . . 11

2. Requirements — M365 & Azure Incident Response & Escalation Runbook (Government / GCC High edition) 12

R1. Responder identities with the right M365 / Azure roles (least privilege) — in the GCC High tenant 12

R2. A defined responder group + on-call rota . . . 12

R3. Access to Microsoft 365 + Azure Service Health (sovereign consoles) . . . 13

R4. Entra ID sign-in & audit log access — plus risk data caveat for GCC High . . . 13

R5. Exchange Online message trace + transport access (sovereign EAC) . . . 13

R6. Defender / security signal access (for security-driven incidents) — and the GCC High parity gaps 14

R7. Azure Monitor / Resource Health access (Azure Government) . . . 14

R8. A Microsoft support plan for GCC High (and knowledge of its severity rules) . . . 14

R9. A ticketing / PSA system as the system of record . . . 15

R10. Defined on-call communication channels + a client status method . . . 15

R11. Defined response-time / SLA targets . . . 15

R12. Change-control & rollback posture for remediations . . . 16

3. Implementation — M365 & Azure Incident Response & Escalation Runbook (Government / GCC High edition) 16

Part A — Stand up the runbook (one-time) 17

A1. Build the Severity & Impact Matrix – Incident – Triage . . . 17
A2. Build the Incident Record Template – Incident – Lifecycle . . . 17
A3. Build the Roles & On-Call Sheet – Incident – Response . . . 18
A4. Build the Client Comms Templates – Incident – Communicate . . . 18
A5. Build the Microsoft Escalation Sheet – Incident – Escalate (GCC High support surface) . . . 19
A6. Build the Diagnostic Playbooks – Incident – Diagnose (GCC High consoles) . . . 19
A7. Build the RCA Template – Incident – Review and the Problem Record – Incident – Permanent Fix
A8. Provision responder access (R1, R2) . . . 22
Part B — The per-incident runbook (every incident) 22
Phase 1 — Detect . . . 22
Phase 2 — Triage . . . 22
Phase 3 — Communicate . . . 22
Phase 4 — Diagnose . . . 23
Phase 5 — Resolve . . . 23
Phase 6 — Escalate to Microsoft (when needed) . . . 23
Phase 7 — RCA → Permanent Fix . . . 24
5. Testing — M365 & Azure Incident Response & Escalation Runbook (Government / GCC High edition) 24
T1. Responder readiness — roles & access (sovereign consoles) *(gate)* . . . 24
T2. Support-plan & escalation readiness (GCC High) *(gate)* . . . 24
T3. Triage correctness *(gate)* . . . 25
T4. Service Health discrimination (sovereign) *(gate)* . . . 25
T5. Communication on cadence *(gate)* . . . 25
T6. Diagnosis reaches root cause (no waiting on absent features) *(gate)* . . . 25
T7. Remediation safety (US Gov endpoints) *(gate)* . . . 26
T8. RCA produced *(gate)* . . . 26
T9. Permanent fix logged *(gate — overall)* . . . 26
Acceptance gate (overall) . . . 26
6. Troubleshooting — M365 & Azure Incident Response & Escalation Runbook (Government / GCC High edition) 27
Sovereign-cloud connectivity (GCC High — read this first) . . . 27
Access & visibility (Detect / Diagnose) . . . 28
Service Health discrimination (Triage) . . . 28
Triage & roles . . . 28
Communication . . . 29
Escalation to Microsoft . . . 29
Remediation . . . 29

RCA → Permanent Fix loop . . . 30
7. Best Practices — M365 & Azure Incident Response & Escalation Runbook (Government / GCC High edition) 30
Sovereign-cloud discipline (GCC High-specific) . . . 30
Triage & prioritization . . . 31
Roles & coordination . . . 31
Client communication . . . 31
Diagnosis . . . 31
Escalation to Microsoft . . . 32
Remediation safety . . . 32
RCA → Permanent Fix loop . . . 32
Program hygiene . . . 32
8. Deliverables Summary — M365 & Azure Incident Response & Escalation Runbook (Government / GCC High edition) 33
Documentation . . . 33
PowerShell scripts . . . 34
Hero image . . . 35
Appendix A — PowerShell Scripts 36

Executive Summary — M365 & Azure Incident Response & Escalation Runbook (Government / GCC High edition)

The business problem

When a high-priority incident hits a Microsoft 365 or Azure estate — mail flow stops, users can't sign in, Teams calls fail, an Azure workload goes down, or an account is compromised — the difference between a 20-minute and a 4-hour outage is rarely the technology. It is whether the team has a **repeatable way to respond**: a clear severity call, named owners, calm client communication, a diagnostic path that reaches the real cause, a way to escalate to Microsoft, and a discipline that stops the same incident from happening again.

On a **GCC High** tenant there is a second trap. GCC High is **not** the Commercial portal behind a different login: the admin consoles are different (`*.us`, not `*.com`), PowerShell must be pointed at US Gov endpoints (`-Environment USGov`), and **some security-operations capabilities lag or are absent** versus Commercial. A team running Commercial habits on a sovereign tenant hits the wrong portals, or waits during a live P1 for an automated action that GCC High doesn't provide. Most teams improvise all of this under pressure — and in a regulated, FedRAMP-High/IL4 estate the cost of improvising is measured in both downtime and compliance exposure.

What this package delivers

A complete, white-label **incident & problem management runbook** for Microsoft 365 GCC High and Azure Government — the operating system for high-priority and escalated incidents on a sovereign tenant. It ships:

- An explicit **Commercial-vs-GCC-High parity map** — every admin URL (`*.us`), every PowerShell endpoint (`-Environment USGov`), and every feature gap that changes the runbook — all **cited to Microsoft** so a responder never guesses.
- A **severity & impact matrix** with response and update-cadence targets (P1–P4).
- The named **incident roles** — Incident Lead, Communications Lead, Scribe — and an on-call model.
- A **triage method** (impact × risk × effort) and a Service-Health-first discipline (in the `*.us` consoles) that separates a Microsoft outage from a tenant fault.
- **Per-workload diagnostic playbooks** with the real **GCC High** console paths — Exchange Online mail flow, Entra ID sign-in/auth, Teams, SharePoint/OneDrive, Intune, and Azure resource health — plus Microsoft Service Health.
- **Client communication templates and a cadence** for the whole incident lifecycle.
- A **Microsoft escalation sheet** (GCC High support surface, honest plan-dependent severity guidance, no phantom Threat Experts lifeline) and a **root-cause analysis** format.
- The **RCA → permanent fix loop** that converts each incident into a lasting hardening change.
- **Five PowerShell tools** — discovery, readiness validation, safe remediation, scope/role targeting, and MTTA/MTTR reporting — all connecting to **US Gov Graph and Azure Government** endpoints.

The phase model

Detect → **Triage** → **Communicate** → **Diagnose** → **Resolve** → **RCA** → **Permanent Fix** — every incident, every time, with named artifacts and explicit gates, executed in the correct sovereign consoles.

Outcomes

- **Lower MTTA/MTTR** — consistent triage and diagnosis instead of improvisation; response no longer depends on who is on call.
- **No sovereign-cloud surprises** — right portals, right endpoints, and no live-incident time lost waiting on a Commercial feature (Threat Experts, AIR) that GCC High doesn't have.
- **Confident clients** — early, factual, on-cadence communication during the worst moments.
- **Real root causes** — diagnostics that reach the *why*, then escalate to Microsoft with evidence when the cause is platform-side.
- **Fewer repeat incidents** — the RCA-to-permanent-fix loop hardens the estate after each event.
- **Auditable, in-boundary managed service** — every incident has a named record, a severity, a timeline, and an RCA, demonstrating mature incident & problem management to clients and to the compliance posture (FedRAMP High / DoD IL4 / DFARS) the tenant is chosen for.

Honest scope

This package is process, playbooks, and tooling — not a guarantee that every outage is locally fixable. When Microsoft is down, its value is correct triage, accurate client communication, and a well-formed escalation. **Parity is a moving target:** the gaps are cited to Microsoft as of authoring and close over time — the runbook tells you to re-verify rather than trusting a static list. Microsoft support severity and response are **plan-dependent** and confirmed in the GCC High support surface; response-time targets ship as sensible defaults and are overridden by your contractual SLAs.

Investment

A premium managed-services enablement package (~$2,495) that pays for itself the first time a P1 on a sovereign tenant is resolved an hour faster — because the team was in the right `*.us` console, connected to the right endpoint, and not waiting on a capability that GCC High never had.

1. Solution Overview — M365 & Azure Incident Response & Escalation Runbook (Government / GCC High edition)

What this solution is

This is a complete **operational runbook** for leading high-priority and escalated incidents across **Microsoft 365 and Azure — on a US sovereign-cloud (GCC High) tenant**. It is an **incident & problem management package** for a managed-services team operating in Microsoft 365 GCC High and Azure Government: it gives a responder everything needed to take a P1/P2 incident from first alert to a verified, permanent fix — the severity/impact matrix, the named roles, the triage method, the per-workload

diagnostic playbooks, the client communication cadence and templates, the path to escalate to Microsoft support, the root-cause analysis (RCA) format, and the post-incident loop that converts a one-off fix into a permanent control.

It is a **playbook, not a wizard.** It pairs a phased incident lifecycle with named artifacts and per-workload diagnostic procedures so an on-call engineer, an incident lead, and a client-facing communicator can run a controlled, auditable response — every time, regardless of which engineer picks up the page.

Why a separate GCC High edition exists. GCC High is **not** the Commercial portal behind a different sign-in. The sovereign-cloud admin URLs are different (`*.us`, not `*.com`), PowerShell must be pointed at US Gov endpoints (`-Environment USGov`), and **some security-operations capabilities lag, are limited, or are absent** versus Commercial. A responder who runs the Commercial runbook on a GCC High tenant will hit the wrong portals, or wait for an automated action that never fires — during a live P1. This edition ships the correct portals, endpoints, and an explicit parity map so the runbook is *accurate for the environment it runs in*.

Commercial vs GCC High — what's different

This is the single most important section for a responder moving from a Commercial tenant. **Confirm each of these before your first incident — never assume a Commercial habit carries over.**

Admin portals (use the `*.us` sovereign URLs)

Console	Commercial URL	GCC High URL (use this)
Sign-in / authentication	`login.microsoftonline.com`	`login.microsoftonline.us`
Microsoft 365 admin center	`admin.microsoft.com`	`portal.office365.us`
Microsoft Entra admin center	`entra.microsoft.com`	`entra.microsoft.us`
Exchange admin center	`admin.exchange.microsoft.com`	`admin.exchange.office365.us`
Microsoft Teams admin center	`admin.teams.microsoft.com`	`admin.gov.teams.microsoft.us`
Microsoft Intune admin center	`intune.microsoft.com`	`intune.microsoft.us`
Microsoft Purview / compliance	`compliance.microsoft.com`	`compliance.microsoft.us`
Microsoft Defender portal	`security.microsoft.com`	`security.microsoft.us`
Azure portal	`portal.azure.com`	`portal.azure.us`

Sources: Microsoft Learn — [Microsoft Defender XDR for US Government](https://learn.microsoft.com/en-us/defender-xdr/usgov); [Microsoft Graph national cloud deployments](https://learn.microsoft.com/en-us/graph/deployments); [MSPortals.io — Government Portals](https://msportals.io/usgovt.html); [Microsoft Teams — GCC High deployments](https://learn.microsoft.com/en-us/microsoftteams/plan-for-government-gcc-high).

PowerShell / API endpoints

- **Microsoft Graph:** service root is `https://graph.microsoft.us` (US Gov L4 / GCC High); connect with `Connect-MgGraph -Environment USGov`. Without `-Environment USGov`, the module authenticates against the Commercial endpoint (`login.microsoftonline.com`) and your GCC High sign-in fails. (Source: [Microsoft Graph national cloud

deployments](https://learn.microsoft.com/en-us/graph/deployments).)

• **Azure PowerShell (Az):** connect with `Connect-AzAccount -Environment AzureUSGovernment`; the Azure portal endpoint is `portal.azure.us`. (Source: [Microsoft Graph national cloud deployments](https://learn.microsoft.com/en-us/graph/deployments).)

• **Defender XDR API:** GCC High uses `https://api-gov.security.microsoft.us` (login via `login.microsoftonline.us`). (Source: [Microsoft Defender XDR for US Government](https://learn.microsoft.com/en-us/defender-xdr/usgov).)

• **Defender for Endpoint API:** GCC High uses `https://api-gov.securitycenter.microsoft.us`. (Source: [Defender for Endpoint for US Government](https://learn.microsoft.com/en-us/defender-endpoint/gov).)

Feature parity gaps that change the runbook

GCC High "uses the same underlying technologies as Microsoft Defender in Azure Commercial" but "doesn't have complete parity with the commercial offering" (Microsoft, [Defender XDR for US Government](https://learn.microsoft.com/en-us/defender-xdr/usgov)). The gaps that materially affect an incident responder:

• **Microsoft Threat Experts — NOT available in GCC High** ("on engineering backlog"). Do not build an escalation step that calls Threat Experts / "Ask a defender expert"; that lifeline does not exist here. (Source: [Defender XDR for US Government](https://learn.microsoft.com/en-us/defender-xdr/usgov), [Defender for Endpoint for US Government](https://learn.microsoft.com/en-us/defender-endpoint/gov).)

• **Automated investigation & response (AIR):** historically available in **Commercial and GCC only** and lagging in GCC High. Do not assume an alert will self-investigate/auto-remediate; the runbook uses **manual** diagnostic playbooks so it is correct whether or not AIR has fired. **Verify AIR status in your own GCC High tenant** before relying on it. (Flag: *verify for GCC High* — availability shifts as Microsoft closes gaps.)

• **Microsoft Defender for IoT (enterprise IoT security) — NOT available** in GCC High. Exclude IoT-device incidents from the Defender-driven workflow. (Source: [Defender XDR for US Government](https://learn.microsoft.com/en-us/defender-xdr/usgov).)

• **Microsoft Secure Score** is available in GCC High, but **some security recommendations are not**; and some **Defender Vulnerability Management** functions (report inaccuracy, request CVE support) are absent. This affects the Permanent-Fix phase where you'd normally track a Secure Score / MDVM recommendation. (Source: [Defender for Endpoint for US Government](https://learn.microsoft.com/en-us/defender-endpoint/gov).)

• **Advanced hunting:** in the **Defender portal**, features in general availability are available in GCC High. Note the documented **GCC (Moderate)** limitation — cross-table queries spanning Microsoft Sentinel *and* Defender XDR aren't supported there — which does **not** apply to GCC High but is a common point of confusion; **verify** your tenant. (Source: [Support for Microsoft Defender XDR connector data types in Microsoft Sentinel for GCC environments](https://learn.microsoft.com/en-us/azure/sentinel/microsoft-365-defender-cloud-support).)

• **Microsoft Sentinel** runs in **Azure Government** for GCC High SOCs; onboard/hunt from the Defender portal `security.microsoft.us` and the Azure Government portal `portal.azure.us`. Some connectors and preview features differ by cloud — **verify connector availability for GCC High** before depending on a data source in an incident. (Source: [Microsoft Sentinel feature

availability](https://learn.microsoft.com/en-us/azure/sentinel/feature-availability).)

> ***Golden rule for this edition:*** *where the runbook names a console, it names the* `*.us` *one. Where a Commercial feature might be absent (Threat Experts, AIR, some Secure Score / MDVM items, IoT), the runbook flags it and gives a* ***manual*** *path so a live incident is never blocked waiting on a capability that isn't in GCC High. When Microsoft's parity docs change, re-verify — the gaps close over time.*

The phase model

Every incident this runbook governs moves through seven phases, used verbatim throughout the package:

Detect → **Triage** → **Communicate** → **Diagnose** → **Resolve** → **RCA** → **Permanent Fix**

- **Detect** — the incident surfaces (monitoring alert, client report, Microsoft Service Health advisory, security signal in `security.microsoft.us`).
- **Triage** — classify severity using the matrix, assign roles, and prioritize using impact × risk × effort.
- **Communicate** — open the client communication cadence with the right template at the right interval.
- **Diagnose** — run the named per-workload diagnostic playbook (in the `*.us` consoles) to reach a root cause (not just a symptom).
- **Resolve** — apply the fix (mitigation first, then full resolution), verify service is restored.
- **RCA** — produce the root-cause analysis record while the timeline is fresh.
- **Permanent Fix** — convert the RCA's corrective actions into a permanent control (a problem record), so the same incident does not recur.

This maps to the standard managed-services lifecycle of **Assess** → **Stabilize** → **Diagnose** → **Remediate** → **Verify** → **Review** → **Expand**: Triage is the Assess gate, Communicate+Resolve stabilize and remediate, RCA is the Review, and Permanent Fix is the Expand step that hardens the estate.

The named artifacts

The package stands up a small set of reusable artifacts, each named once and reused verbatim across the runbook, scripts, and client comms:

Artifact	Purpose
Severity & Impact Matrix – Incident – Triage	The single classification table mapping P1–P4 to impact, urgency, and response/cadence targets.
Incident Record Template – Incident – Lifecycle	The master record for one incident: timeline, severity, roles, actions, decisions, MTTA/MTTR.
Roles & On-Call Sheet – Incident – Response	Names the Incident Lead, Communications Lead, and Scribe per incident and the on-call rota.
Client Comms Templates – Incident – Communicate	Initial / Update / Resolution / Post-Incident message templates for clients and a status method.
Microsoft Escalation Sheet – Incident – Escalate	How and when to open and severity-set a Microsoft support case in the **GCC High** support surface (plan-dependent).

Artifact	Purpose
RCA Template – Incident – Review	The structured root-cause analysis (timeline, root cause, contributing factors, corrective actions).
Problem Record – Incident – Permanent Fix	Tracks the permanent control derived from the RCA through to closure.
GCC High Parity Notes – Incident – Sovereign	The cited reference of Commercial-vs-GCC-High portal/endpoint/feature differences (see `gcc-high-parity-notes.md`).
Diagnostic Playbooks – Incident – Diagnose	The per-workload procedures (Exchange Online, Entra ID, Teams, SharePoint/OneDrive, Intune, Azure) using the `*.us` consoles.

The per-workload diagnostic playbooks

The Diagnose phase is driven by named, per-workload playbooks, each with the **real GCC High console paths** an engineer follows:

1. **Exchange Online Mail Flow Playbook – Diagnose – Exchange** — message trace, transport rules, connectors, queues (`admin.exchange.office365.us`).
2. **Entra Sign-in & Auth Playbook – Diagnose – Identity** — sign-in logs, Conditional Access, MFA, risk (`entra.microsoft.us`).
3. **Teams Service Playbook – Diagnose – Teams** — call quality, meeting/chat health, Teams admin diagnostics (`admin.gov.teams.microsoft.us`).
4. **SharePoint & OneDrive Playbook – Diagnose – Collaboration** — site/sync health, sharing, storage.
5. **Intune Device & App Playbook – Diagnose – Endpoint** — enrollment, policy/app delivery, compliance (`intune.microsoft.us`).
6. **Azure Resource Health Playbook – Diagnose – Azure** — Resource Health, Service Health, Azure Monitor (`portal.azure.us`).
7. **Microsoft Service Health Playbook – Diagnose – Platform** — distinguish a Microsoft outage from a tenant issue (`portal.office365.us` + `portal.azure.us`).

How it works (the flow)

1. **Detect** the incident and open an **Incident Record Template – Incident – Lifecycle**.
2. **Triage**: score with the **Severity & Impact Matrix – Incident – Triage**; assign roles from the **Roles & On-Call Sheet – Incident – Response**.
3. **Check Microsoft Service Health first** — confirm whether this is a Microsoft-side outage before deep diagnosis (in the `*.us` consoles).
4. **Communicate** to the client using the matching **Client Comms Templates – Incident – Communicate** at the cadence the severity demands.

5. **Diagnose** with the right **Diagnostic Playbook – Incident – Diagnose** until a root cause is identified — checking the **GCC High Parity Notes** if a Commercial capability appears missing.

6. **Escalate to Microsoft** via the **Microsoft Escalation Sheet – Incident – Escalate** if root cause is Microsoft-side or beyond local control.

7. **Resolve**: mitigate, then fully restore; verify; send the Resolution comms.

8. **RCA**: complete the **RCA Template – Incident – Review** within the agreed window.

9. **Permanent Fix**: open a **Problem Record – Incident – Permanent Fix** and drive the corrective control to closure.

Benefits

• **Faster, more consistent response** — every engineer runs the same triage, comms, and diagnostics *in the correct sovereign consoles*, so MTTA/MTTR drop and don't depend on who is on call.

• **No sovereign-cloud surprises** — the parity map means no engineer wastes a live P1 waiting on a Commercial feature (Threat Experts, AIR) that GCC High doesn't have.

• **Clear client communication** — the templates and cadence remove the "what do we tell the client?" hesitation under pressure.

• **Root cause, not just symptom** — the playbooks push past restarting a thing toward *why* it broke.

• **No repeat incidents** — the RCA → Permanent Fix loop converts each incident into a hardening change.

• **Auditable** — every incident has a named record, a severity, a timeline, and an RCA — usable as managed-service evidence and for the compliance posture (FedRAMP High / DoD IL4 / DFARS) a GCC High tenant is chosen for.

Honest limitations

• **This is process and tooling, not a fix for every outage.** When the root cause is a Microsoft-side outage, the runbook's job is correct triage, accurate client comms, and escalation — not making Microsoft's service come back faster.

• **Parity is a moving target.** The gaps documented here are cited to Microsoft as of authoring; Microsoft closes gaps over time. **Re-verify the parity notes** for your tenant rather than trusting a static list.

• **Microsoft support severity and response are plan-dependent** and confirmed in the **GCC High** support surface — not the Commercial one. The runbook tells you how to check yours.

• **Diagnostics require the right access.** Reaching root cause depends on having the reader/admin roles and log access in Requirements R1–R12 *in the GCC High tenant*; the runbook will stall at Diagnose without them.

The standard runbook model

This solution follows the **standard incident-management lifecycle** — a phased model with named artifacts, explicit gates, and a feedback loop:

Detect → Triage → Communicate → Diagnose → Resolve → RCA → Permanent Fix.

Each phase has an entry trigger, an exit criterion, and named deliverables. Triage gates whether the response escalates; RCA gates closure; and Permanent Fix gates whether the incident is allowed to recur. No incident is "closed" until its RCA is produced and any required Problem Record is opened.

2. Requirements — M365 & Azure Incident Response & Escalation Runbook (Government / GCC High edition)

Each requirement is structured as **Description / Why / Verify / Implement-if-missing**. **Never assume a prerequisite exists** — verify each one before you rely on the runbook in a live incident. Requirements are numbered R1–R12. These layer on top of the shared `global-requirements-managed.md`.

> ***Sovereign-cloud note:*** *every *Verify* step below uses the **GCC High (**`*.us`**)** consoles and **US Gov** PowerShell endpoints (`Connect-MgGraph -Environment USGov`, `Connect-AzAccount -Environment AzureUSGovernment`). If a step sends you to a `*.com` Commercial URL by habit, you are in the wrong cloud. See `gcc-high-parity-notes.md` for the full cited portal map.*
>
> *Naming convention used throughout the runbook (reused verbatim in implementation): every created object is named `<Purpose> – <Scope> – <Stage>`, e.g. `Responder Access Group – Identity – Response`.*

R1. Responder identities with the right M365 / Azure roles (least privilege) — in the GCC High tenant

• **Description:** Each responder has a working account **in the GCC High tenant** with the roles needed to **diagnose and act** across the in-scope workloads — at minimum reader-level visibility everywhere, with scoped admin where remediation is expected (Exchange Administrator, Teams Administrator, SharePoint Administrator, Intune Administrator, Azure Reader/Contributor, and a Security Reader).

• **Why:** The Diagnose phase stalls without log and console access; the Resolve phase stalls without scoped admin. Personal Global Admin for everyone is the wrong answer (blast radius, audit). Reader-first with scoped, PIM-eligible admin is the target — and it must exist in the *sovereign* tenant, not a Commercial one.

• **Verify:** In **Entra admin center** (`https://entra.microsoft.us`) → **Roles & admins**, confirm each responder holds the expected role(s). In **Azure portal** (`https://portal.azure.us`) → **Subscriptions** → **Access control (IAM)**, confirm at least **Reader** on in-scope subscriptions.

• **Implement-if-missing:** Create the responder access group (R2) and assign the minimum roles; prefer **PIM eligible** activation over standing access. Run `scripts\INCIDENTG-Set-ScopeTargeting.ps1 -WhatIf` first (it uses `-Environment USGov`), then apply.

R2. A defined responder group + on-call rota

• **Description:** A named Entra group `Responder Access Group – Identity – Response` containing the response team, and a documented on-call rota covering the hours your SLAs require.

• **Why:** Role assignment, alert routing, and "who is the Incident Lead right now?" all key off a known group and rota. An incident at 02:00 must have a named owner.

• **Verify:** In **Entra admin center** (`https://entra.microsoft.us`) → **Groups**, confirm the group exists with current members. Confirm a published on-call schedule (PSA/calendar/paging tool) names the current on-call engineer.

• **Implement-if-missing:** Create the group (`INCIDENTG-Set-ScopeTargeting.ps1`), populate it, and stand up an on-call rota in your paging/PSA tool. Record both in the **Roles & On-Call Sheet – Incident – Response**.

R3. Access to Microsoft 365 + Azure Service Health (sovereign consoles)

• **Description:** Responders can read **Service health** in the Microsoft 365 admin center and **Service Health** in the Azure portal — the `*.us` ones.

• **Why:** The very first diagnostic question — "is this Microsoft or is this us?" — is answered here. Misclassifying a Microsoft outage as a tenant fault wastes the response.

• **Verify:** Open **Microsoft 365 admin center** (`https://portal.office365.us`) → **Health** → **Service health** and confirm it loads (requires Global Reader, Service Support Admin, or higher). Open **Azure portal** (`https://portal.azure.us`) → **Service Health** and confirm advisories load.

• **Implement-if-missing:** Assign **Service Support Administrator** or **Global Reader** to responders; confirm the account can see **Health** → **Service health** and the Azure **Service Health** blade in the sovereign consoles.

R4. Entra ID sign-in & audit log access — plus risk data caveat for GCC High

• **Description:** Responders can read **Sign-in logs** and **Audit logs** in Entra (`entra.microsoft.us`) and, where licensed, **Risky sign-ins / Risk detections** (Identity Protection).

• **Why:** Identity is the most common root cause of "can't log in / can't get mail" incidents. Sign-in logs show failure reasons, Conditional Access results, and MFA outcomes; audit logs show what changed.

• **Verify: Entra admin center** (`https://entra.microsoft.us`) → **Monitoring** → **Sign-in logs** and **Monitoring** → **Audit logs** both load. For risk data, **Protection** → **Identity Protection** requires **Entra ID P2** (in GCC High this is delivered via the **G5 Gov** stack / EMS E5 for GCC High). **Verify Identity Protection availability and behavior for GCC High in your tenant** — do not assume Commercial risk-detection parity.

• **Implement-if-missing:** Assign a role that grants log read (**Reports Reader**, **Security Reader**, or **Global Reader**). For risk detections, confirm the P2 entitlement in the GCC High licensing stack; if absent, document that risk-based diagnosis is unavailable and rely on raw sign-in logs.

R5. Exchange Online message trace + transport access (sovereign EAC)

• **Description:** Responders can run **message trace** and read transport rules, connectors, and queues in the Exchange admin center, plus connect via the Exchange Online PowerShell module **to the US Gov environment**.

• **Why:** Mail-flow incidents (NDRs, delays, blocked senders, connector failures) are diagnosed through message trace and transport configuration — there is no other reliable path.

• **Verify: Exchange admin center** (`https://admin.exchange.office365.us`) → **Mail flow** → **Message trace** runs; → **Mail flow** → **Rules / Connectors** load. `Connect-ExchangeOnline -ExchangeEnvironmentName O365USGovGCCHigh` succeeds for the responder account.

• **Implement-if-missing:** Assign **Exchange Administrator** (or a custom RBAC role with View-Only Recipients + Message Tracking + Transport read). Install `ExchangeOnlineManagement` on the responder workstation and always connect with the `O365USGovGCCHigh` environment name.

R6. Defender / security signal access (for security-driven incidents) — and the GCC High parity gaps

• **Description:** Responders can read alerts and incidents in the **Microsoft Defender portal** (`security.microsoft.us`) — Defender for Office 365 / Endpoint / Identity, as licensed.

• **Why:** Many P1s start as a security signal (account compromise, mass mail, malware). The runbook needs to read these alerts to triage and to feed the Diagnose phase.

• **Verify: Microsoft Defender portal** (`https://security.microsoft.us`) → **Incidents & alerts** → **Incidents** loads for the responder account.

• **Implement-if-missing:** Assign **Security Reader** (read) or **Security Administrator** (act); confirm the relevant Defender workloads are licensed and onboarded. **Known GCC High parity gaps to account for (cited): Microsoft Threat Experts is NOT available** in GCC High; **Automated investigation & response (AIR)** historically lags (verify in your tenant); **Defender for IoT enterprise IoT security is NOT available**. Do not design escalation or Resolve steps that depend on these. See `gcc-high-parity-notes.md`.

R7. Azure Monitor / Resource Health access (Azure Government)

• **Description:** Responders can read **Resource Health** and **Service Health** per resource and query **Azure Monitor** (Metrics, Log Analytics) for in-scope Azure Government workloads.

• **Why:** Azure resource incidents (VM down, App Service 5xx, SQL throttling) are diagnosed through Resource Health and Monitor metrics/logs.

• **Verify: Azure portal** (`https://portal.azure.us`) → a resource → **Resource health** loads; **Monitor** → **Metrics** and any **Log Analytics workspace** are queryable by the responder (**Monitoring Reader** or **Reader**). In PowerShell, `Connect-AzAccount -Environment AzureUSGovernment` succeeds.

• **Implement-if-missing:** Assign **Monitoring Reader** (and **Reader**) on in-scope subscriptions/resources; confirm diagnostic settings send logs to a Log Analytics workspace so history exists at incident time.

R8. A Microsoft support plan for GCC High (and knowledge of its severity rules)

• **Description:** A known Microsoft support entitlement for escalation **for the GCC High tenant** — at minimum the support included with the subscription, ideally **Microsoft Unified** (the successor to Premier) — and documented knowledge of which case severities your plan permits and their response commitments.

• **Why:** When root cause is Microsoft-side or beyond local control, you escalate to Microsoft **through the GCC High support surface**. **The severities you can set and the response you receive are plan-dependent** — included support, Microsoft Unified, and legacy Premier differ. You must know yours *before* the incident, not during it.

• **Verify: Microsoft 365 admin center** (`https://portal.office365.us`) → **Support** → **Help & support** shows your entitlement and the severity options available when opening a case. Azure: **Azure portal** (`https://portal.azure.us`) → **Help + support** → **Create a support request** shows the **severity** levels your plan allows (e.g. Sev A/B/C; Sev A typically requires a paid Azure support plan). **Confirm the support path is the GCC High one**, not a Commercial tenant's.

• **Implement-if-missing:** Confirm or purchase the appropriate support plan (Microsoft Unified for managed-service breadth, or an **Azure support plan** for Sev A response) for the GCC High tenant. Record entitlement details and severity rules in the **Microsoft Escalation Sheet – Incident – Escalate**.

R9. A ticketing / PSA system as the system of record

• **Description:** A ticketing or PSA platform where every incident is logged, timestamped, and tracked through closure.

• **Why:** The **Incident Record Template – Incident – Lifecycle**, MTTA/MTTR metrics, and the audit trail all need a durable home. Email and chat are not a system of record.

• **Verify:** Confirm a ticket type/queue exists for incidents and that responders can create and update tickets with timestamps.

• **Implement-if-missing:** Configure an incident ticket type in your PSA carrying the fields the Incident Record Template needs (severity, detected/acknowledged/resolved times, roles, RCA link). Map the template to those fields. *(For regulated GCC High engagements, confirm the PSA's own data-residency/compliance posture meets the contract.)*

R10. Defined on-call communication channels + a client status method

• **Description:** A real-time responder channel (e.g. a dedicated chat/bridge for war-rooming) **and** a defined method to communicate status to clients — a status page, an email distribution, or a documented call/contact path per client.

• **Why:** The Communicate phase requires both an internal coordination channel and an outward client channel. Improvising the client channel mid-incident erodes trust. *(In GCC High, confirm the war-room tool itself is an approved sovereign service where the contract requires it — a Commercial chat tool may be out of scope.)*

• **Verify:** Confirm a standing incident bridge/channel exists and that each managed client has a documented status/comms method and an after-hours contact.

• **Implement-if-missing:** Create the incident war-room channel; for each client, record the comms method and emergency contact in the **Client Comms Templates – Incident – Communicate** front matter.

R11. Defined response-time / SLA targets

• **Description:** Documented response and update-cadence targets per severity — either your contractual SLAs or the matrix defaults adopted as policy.

• **Why:** Triage and the comms cadence are meaningless without targets; "P1 = update every 30 minutes" must be a stated commitment, not an ad-hoc decision.

• **Verify:** Confirm the **Severity & Impact Matrix – Incident – Triage** carries response and cadence targets and that these match (or exceed) any client SLAs.

• **Implement-if-missing:** Populate the matrix targets from contracts; where no contract specifies, adopt the defaults in §03 as internal policy.

R12. Change-control & rollback posture for remediations

• **Description:** A lightweight change path for incident remediations, including approval for high-risk fixes and a back-out for each, plus the discipline to record changes in the incident record.

• **Why:** Incident fixes are applied under pressure; an un-recorded or irreversible change can turn one incident into two. Emergency change still needs a record and a back-out — and in a regulated GCC High estate, an auditable change trail is part of the compliance obligation, not a nicety.

• **Verify:** Confirm an **emergency change** path exists and that responders know when a fix needs approval vs. when to act-then-document. Confirm `INCIDENTG-Remediation.ps1` defaults to `-WhatIf`.

• **Implement-if-missing:** Define an emergency-change rule (what may be done without prior approval, what may not), and require every remediation to be logged in the **Incident Record Template – Incident – Lifecycle**. See also `global-requirements-managed.md`.

3. Implementation — M365 & Azure Incident Response & Escalation Runbook (Government / GCC High edition)

This is the operational build-out **and** the live runbook, authored for a **Microsoft 365 GCC High + Azure Government** tenant. Part A stands up the named artifacts once (the matrix, templates, playbooks, escalation sheet, RCA, and responder access). Part B is the per-incident runbook you execute every time, phase by phase, with full **GCC High (`*.us`) console navigation paths**.

Steps are imperative and ordered. Every created object uses the naming standard `<Purpose> – <Scope> – <Stage>` verbatim. **Never assume a prerequisite exists** — each step that depends on access points back to a Requirement (R1–R12) to verify and implement-if-missing.

> ***Sovereign-cloud reminder (read once, apply everywhere below):*** *every console path in this runbook uses the `*.us` URL and every PowerShell connection uses a **US Gov** endpoint. Quick reference (full cited map in `gcc-high-parity-notes.md`):* | *Console* | *GCC High URL* | |---|---| | *Sign-in* | `login.microsoftonline.us` | | *M365 admin* | `portal.office365.us` | | *Entra admin* | `entra.microsoft.us` | | *Exchange admin* | `admin.exchange.office365.us` | | *Teams admin* | `admin.gov.teams.microsoft.us` | | *Intune admin* | `intune.microsoft.us` | | *Purview / compliance* | `compliance.microsoft.us` | | *Defender portal* | `security.microsoft.us` | | *Azure portal* | `portal.azure.us` | *PowerShell:* `Connect-MgGraph -Environment USGov` · `Connect-AzAccount -Environment AzureUSGovernment` · `Connect-ExchangeOnline -ExchangeEnvironmentName O365USGovGCCHigh`.

Part A — Stand up the runbook (one-time)

A1. Build the Severity & Impact Matrix – Incident – Triage

1. Create the artifact **Severity & Impact Matrix – Incident – Triage** in your PSA/knowledge base (R9). It is a table mapping **severity (P1–P4)** against **business impact**, **urgency**, **response-time target**, and **update cadence**.

2. Populate it from your client SLAs (R11). Where no SLA specifies, adopt these defaults:

Severity	Definition (impact)	Examples	Response target	Update cadence
P1 — Critical	Total or near-total outage of a critical service for many/all users; no workaround.	Mail flow stopped tenant-wide; all users can't sign in; suspected active account compromise.	Immediate (≤15 min)	Every 30 min
P2 — High	Major degradation or outage for a department/site, or a critical service degraded with a partial workaround.	One site can't access SharePoint; Teams calls failing for a region; recurring sign-in failures for a group.	≤30 min	Every 60 min
P3 — Moderate	Single user or small group impacted; workaround exists; not business-critical.	One mailbox delayed; one device won't enroll.	≤4 business hours	Daily
P4 — Low	Minor issue, question, or cosmetic; no business impact.	Single non-urgent config query.	≤1 business day	On change

1. **Verify:** confirm the matrix carries response and cadence targets and that they match or exceed every client SLA (R11). Store one canonical copy; reference it from the Incident Record Template.

Note (severity ≠ Microsoft severity): *these P1–P4 levels are *your* internal/client severities. Microsoft support uses its own severity scheme (Azure Sev A/B/C; M365 Critical/Severe/etc.), and which you may set is plan-dependent and confirmed in the **GCC High** support surface — see A5 and Part B Phase 6.*

A2. Build the Incident Record Template – Incident – Lifecycle

1. In your PSA (R9), create the incident ticket type **Incident Record Template – Incident – Lifecycle** with these fields:

• Incident ID, Title, Client/Tenant, **Severity** (from A1).

• **Timestamps:** Detected, Acknowledged (drives MTTA), Mitigated, Resolved (drives MTTR), Closed.

• **Roles:** Incident Lead, Communications Lead, Scribe (from A3).

• **Timeline / running log** (every action and decision, timestamped — the Scribe owns this).

• **Affected workload(s)** and the Diagnostic Playbook used.

• **Microsoft case number** (if escalated) and Microsoft severity — recorded against the **GCC High** support entitlement.

• **Root cause** (link to RCA), **Corrective actions**, **Problem Record** link.

1. **Verify:** create a test incident and confirm all fields capture and timestamp correctly.

A3. Build the Roles & On-Call Sheet – Incident – Response

1. Create **Roles & On-Call Sheet – Incident – Response** documenting the three per-incident roles and the on-call rota (R2):

• **Incident Lead** — owns the response, makes decisions, sets severity, decides escalation. The single throat to choke.

• **Communications Lead** — owns all client and stakeholder comms (A4); shields the Lead and engineers from status requests.

• **Scribe** — maintains the timeline in the Incident Record; captures who-did-what-when for the RCA.

1. For a small team, one person may hold multiple roles — but the **Incident Lead role is always named** before diagnosis begins.

2. Link the on-call rota (R2) so "current Incident Lead" is always resolvable.

3. **Verify:** confirm the current on-call engineer is named and reachable through the paging tool.

A4. Build the Client Comms Templates – Incident – Communicate

1. Create **Client Comms Templates – Incident – Communicate** with four templates and a defined client status method (R10):

Initial notification (sent at first triage):

> **Subject: [P{n}] Service incident affecting {service} — investigating* We are aware of an issue affecting {service} that began at approximately {time}. Impact: {who/what}. Our team is investigating and treating this as Priority {n}. Next update by {time + cadence}.*

Update (sent every cadence interval):

> **Subject: [P{n}] {service} incident — update {hh:mm}* Status: {investigating / mitigated / monitoring}. What we know: {brief}. What we're doing: {brief}. Workaround (if any): {…}. Next update by {time}.*

Resolution:

> **Subject: [P{n}] {service} incident — RESOLVED* Service was restored at {time}. Cause (preliminary): {brief}. We are completing a full root-cause analysis and will share findings by {date}. Thank you for your patience.*

Post-incident summary (sent after RCA):

> *Subject: Post-incident report — {service} incident {date}* Summary, timeline, root cause, corrective actions, and the permanent fix we are implementing. (Body = the client-facing extract of the RCA Template.)*

1. Record each client's **status method** (status page / email DL / call path) and after-hours contact (R10) in the template front matter. For regulated clients, note any requirement that incident comms stay within an approved channel.

2. **Verify:** send a test Initial+Update to an internal address; confirm the cadence reminder fires.

A5. Build the Microsoft Escalation Sheet – Incident – Escalate (GCC High support surface)

1. Create **Microsoft Escalation Sheet – Incident – Escalate** recording, for **each tenant/subscription**:

• The Microsoft **support entitlement** (included support / **Microsoft Unified** / legacy Premier / **Azure support plan**) — confirm at **Microsoft 365 admin center** (`https://portal.office365.us`) → **Support** → **Help & support**, and **Azure portal** (`https://portal.azure.us`) → **Help + support** (R8). *Confirm you are in the GCC High support path, not a Commercial tenant's.*

• The **case severities your plan permits** and their response commitments. *Be explicit that this is plan-dependent:* Azure **Sev A** (critical, fastest response) typically requires a **paid Azure support plan**; M365 case severity options likewise vary by entitlement. Do not promise a severity your plan can't set.

• **How to open a case:**

• **M365 (GCC High):** `https://portal.office365.us` → **Support** → **Help & support** → **Contact support** (or "New service request"); attach the message trace / sign-in log / correlation IDs you gathered in Diagnose.

• **Azure Government:** `https://portal.azure.us` → **Help + support** → **Create a support request**; pick the resource, **Issue type**, and the **severity** your plan allows.

• **Do NOT list "Microsoft Threat Experts / Ask a Defender expert" as an escalation path** — it is **not available in GCC High** (see A6/parity notes). The escalation path is a Microsoft **support case**, plus your internal chain.

• The **internal escalation chain** (when the Incident Lead escalates to a senior engineer / management) and the trigger conditions.

1. **Verify:** confirm each tenant's entitlement and the severity list shown when you begin (do not submit) a case in the `*.us` portals; record screenshots/notes.

A6. Build the Diagnostic Playbooks – Incident – Diagnose (GCC High consoles)

Author the seven named per-workload playbooks below — each is a short, ordered procedure with the **exact GCC High console path**. These are referenced verbatim in Part B Phase 4.

*A6.1 Microsoft Service Health Playbook – Diagnose – Platform *(run this FIRST)**

1. **M365:** `https://portal.office365.us` → **Health** → **Service health**. Review active issues/advisories for the affected workload; note the **issue ID** and status.

2. **Azure:** `https://portal.azure.us` → **Service Health** → **Service issues** (and **Health advisories**, **Security advisories**, **Planned maintenance**). Filter to your subscriptions/regions (US Gov regions).

3. If an active Microsoft issue matches the symptom, this is a **Microsoft-side incident** — switch to "track + communicate + escalate" (Part B Phases 3 & 6), not deep local diagnosis.

A6.2 Exchange Online Mail Flow Playbook – Diagnose – Exchange

1. `https://admin.exchange.office365.us` → **Mail flow** → **Message trace**. Run a trace for the affected sender/recipient and time window; read the delivery status / failure reason.

2. **Mail flow** → **Rules** — check for a transport rule blocking/redirecting the message.

3. **Mail flow** → **Connectors** — check inbound/outbound connector health (relevant to hybrid/third-party gateways).

4. **Defender:** `https://security.microsoft.us` → **Email & collaboration** → **Threat Explorer / Quarantine** — confirm the message wasn't quarantined or ZAP'd.

5. For deeper data: `Connect-ExchangeOnline -ExchangeEnvironmentName O365USGovGCCHigh` (R5) then `Get-MessageTrace` / `Get-MessageTraceDetail`.

A6.3 Entra Sign-in & Auth Playbook – Diagnose – Identity

1. `https://entra.microsoft.us` → **Monitoring** → **Sign-in logs**. Filter to the affected user/time; open a failed sign-in and read **Failure reason**, **Conditional Access** result, and **MFA** detail.

2. **Protection** → **Conditional Access** → **Policies** — check whether a CA policy is blocking; use **What If** to simulate.

3. **Protection** → **Identity Protection** → **Risky sign-ins / Risk detections** (requires **Entra ID P2** via the G5 Gov stack, R4) — check for risk-driven blocks or compromise. **GCC High caveat:** verify Identity Protection risk-detection behavior in your tenant; if P2/risk data is unavailable, fall back to raw sign-in logs.

4. **Monitoring** → **Audit logs** — see recent changes (policy, user, app) that could have caused the regression.

A6.4 Teams Service Playbook – Diagnose – Teams

1. `https://admin.gov.teams.microsoft.us` → **Users** → select the user → **Meetings/Call history**; for quality, open a specific call.

2. **Analytics & reports** → **Call quality dashboard (CQD)** — assess call/meeting quality trends.

3. Confirm underlying dependencies: Teams relies on Exchange (calendar) and SharePoint (files) — cross-check A6.2 and A6.5 if those are implicated.

4. Check **Service health** (A6.1) — Teams incidents are frequently platform-side.

A6.5 SharePoint & OneDrive Playbook – Diagnose – Collaboration

1. From `https://portal.office365.us` → open the **SharePoint admin center** (the GCC High SharePoint admin URL is tenant-specific, of the form `https://<tenant>-admin.sharepoint.us`)

→ **Sites** → **Active sites** — check the affected site's status and storage.

2. For sync issues: on the device, OneDrive sync health and the **Sync admin reports** in the SharePoint admin center.

3. **Sharing:** site/tenant external-sharing settings if access-denied is the symptom. *(External sharing in GCC High is commonly locked down by policy — confirm the sharing posture before treating a deliberate block as a fault.)*

4. Check **Service health** (A6.1) for SharePoint/OneDrive advisories.

A6.6 Intune Device & App Playbook – Diagnose – Endpoint

1. `https://intune.microsoft.us` → **Devices** → **All devices** → select the device → **Device compliance** and **Managed Apps**; review last check-in.

2. **Devices** → **Monitor** → **Enrollment failures** for enrollment incidents.

3. **Apps** → **Monitor** → **App install status** for delivery failures.

4. **Reports** → **Endpoint analytics** for fleet-wide signals.

A6.7 Azure Resource Health Playbook – Diagnose – Azure

1. `https://portal.azure.us` → the affected resource → **Resource health** — read current/historical health and Microsoft's stated cause (platform vs. customer-initiated).

2. **Monitor** → **Metrics** — chart the resource's key metrics around the incident time; **Monitor** → **Logs** (Log Analytics) for diagnostic logs (R7).

3. **Service Health** (A6.1) — confirm whether a regional **Azure Government** issue is in play.

4. For activity/changes: the resource's **Activity log** — see who changed what.

Verify (A6): walk each playbook against a known-good resource and confirm every console path resolves with the responder's access (R3–R7) in the `*.us` consoles.

> ***GCC High diagnosis caveat (apply across A6):*** *do not wait for* ***Automated investigation & response (AIR)*** *to enrich or auto-remediate an alert — AIR historically lags in GCC High; treat these playbooks as your* ***manual*** *primary path and verify AIR status separately. Do not look for a* ***Threat Experts*** *insight — that capability is* ***not available*** *in GCC High. (See* `gcc-high-parity-notes.md`*.)*

A7. Build the RCA Template – Incident – Review and the Problem Record – Incident – Permanent Fix

1. Create **RCA Template – Incident – Review** with sections: **Summary**, **Impact** (who/what/duration), **Timeline** (from the Incident Record), **Root cause** (the *why*, via 5-Whys), **Contributing factors**, **What went well / what didn't**, **Corrective actions** (each with owner + due date), and **Detection/Prevention improvements**.

2. Create **Problem Record – Incident – Permanent Fix** to track each corrective action that is a *permanent control* (not a one-off fix) through to closure, linked back to the RCA and the originating incident. *Where a permanent fix would normally be a Secure Score / Defender Vulnerability Management recommendation, note that some of those items are absent in GCC High (R6) — track the control directly instead.*

3. **Verify:** confirm both templates link bidirectionally to the Incident Record (A2).

A8. Provision responder access (R1, R2)

1. Run `scripts\INCIDENTG-Set-ScopeTargeting.ps1 -WhatIf` to preview creation of `Responder Access Group – Identity – Response` and the reader/scoped-admin role assignments (the script connects with `-Environment USGov`).

2. Review the preview; then re-run with `-Apply` to create the group and assign roles. Prefer **PIM-eligible** for admin roles.

3. Run `scripts\INCIDENTG-Validation.ps1` to confirm responder readiness against the GCC High endpoints: roles present, log access, support plan — expect **PASS** on all checks (R1–R8).

4. **Verify:** sign in as a responder (`login.microsoftonline.us`) and confirm each console path in A6 loads.

Part B — The per-incident runbook (every incident)

Phase 1 — Detect

1. The incident surfaces via a monitoring alert, a client report, a Microsoft Service Health advisory, or a security signal (Defender at `security.microsoft.us`).

2. **Acknowledge** immediately and open an **Incident Record Template – Incident – Lifecycle** (A2). Record **Detected** and **Acknowledged** timestamps (these drive MTTA).

3. Capture the raw symptom verbatim and the reporter/source.

Phase 2 — Triage

1. The on-call engineer assumes (or assigns) the **Incident Lead** role and, for P1/P2, names a **Communications Lead** and **Scribe** from the **Roles & On-Call Sheet – Incident – Response** (A3).

2. Classify severity with the **Severity & Impact Matrix – Incident – Triage** (A1) using **impact × urgency**. To prioritize among competing actions, weigh **impact × risk × effort** — do the highest-impact, lowest-risk, lowest-effort mitigation first.

3. **Run the Microsoft Service Health Playbook – Diagnose – Platform (A6.1) now** — answer "is this Microsoft or us?" before committing to deep diagnosis, using the `*.us` Service Health consoles.

4. Record severity and roles in the Incident Record. This is the gate: P1/P2 trigger the full response; P3/P4 follow the lighter cadence.

Phase 3 — Communicate

1. The **Communications Lead** sends the **Initial notification** from **Client Comms Templates – Incident – Communicate** (A4) via the client's status method, at the severity's response target (R11).

2. Set a recurring reminder at the matrix **update cadence** and send the **Update** template each interval — even "no change yet" is an update.

3. Keep internal coordination in the incident war-room channel (R10); keep client comms calm, factual, and free of internal jargon.

Phase 4 — Diagnose

1. Select the **Diagnostic Playbook – Incident – Diagnose** matching the affected workload (A6.2–A6.7) and follow its ordered `*.us` console steps.

2. Drive toward **root cause**, not symptom — capture correlation IDs, message-trace IDs, sign-in error codes, and Resource Health statements; the Scribe logs each finding with a timestamp.

3. If multiple workloads are implicated, run the dependent playbooks (e.g. Teams → Exchange/SharePoint).

4. If the playbook points at a Microsoft-side cause, proceed to Phase 6 (escalate) in parallel with continued communication.

5. (Optional) Run `scripts\INCIDENTG-Discovery.ps1` to snapshot current Service Health, recent high-severity sign-ins/risk, and Azure resource health in one pass to seed diagnosis — it connects to the **US Gov Graph and Azure Government** endpoints.

6. **GCC High check:** if you expected an alert to have been auto-investigated (AIR) or to carry a Threat Experts note and it hasn't, that is expected here — **do not stall** waiting for it. Continue the manual playbook. (See `gcc-high-parity-notes.md`.)

Phase 5 — Resolve

1. **Mitigate first** (restore service / workaround), **then** fix fully — record both in the Incident Record with timestamps (**Mitigated**, then **Resolved**, which drives MTTR).

2. For common, well-understood fixes, use `scripts\INCIDENTG-Remediation.ps1` — it connects with `-Environment USGov`, defaults to `-WhatIf`; review the preview, then `-Apply` (e.g. revoke a compromised user's sessions; re-enable a disabled mail-flow connector for a *named* object). High-risk fixes follow the emergency-change rule (R12).

3. **Verify the fix** against the original symptom (re-run the relevant playbook step; confirm with the reporter).

4. The Communications Lead sends the **Resolution** template (A4).

Phase 6 — Escalate to Microsoft (when needed)

1. Escalate when root cause is **Microsoft-side** (confirmed in Service Health) or **beyond local control**, using the **Microsoft Escalation Sheet – Incident – Escalate** (A5).

2. Open the case at the right **GCC High** portal (M365 `portal.office365.us` / Azure `portal.azure.us`) and set the **severity your plan permits** — *remember severity options and response are plan-dependent* (R8). Attach the diagnostic evidence gathered in Phase 4.

3. Record the **Microsoft case number** and severity in the Incident Record; keep the client updated that an escalation is in progress. **Note:** there is no Threat Experts lifeline in GCC High — a Microsoft support case is the escalation.

Phase 7 — RCA → Permanent Fix

1. Within the agreed window (e.g. ≤5 business days for P1), complete the **RCA Template – Incident – Review** (A7) from the Incident Record timeline; use **5-Whys** to reach the true root cause.

2. Send the **Post-incident summary** (A4) — the client-facing extract of the RCA.

3. For every corrective action that is a *permanent control*, open a **Problem Record – Incident – Permanent Fix** (A7) with an owner and due date, and drive it to closure (a config hardening, a new monitor/alert, a policy change, a documentation/runbook update). Where you'd normally cite a Secure Score / MDVM recommendation that is absent in GCC High (R6), track the control directly.

4. Run `scripts\INCIDENTG-Report.ps1` to compute **MTTA/MTTR** and append this incident to the trend metrics.

5. **Close** the incident only when the RCA exists and any required Problem Record is open. The loop is complete when the permanent fix lands — converting this incident into estate hardening (the Expand step).

5. Testing — M365 & Azure Incident Response & Escalation Runbook (Government / GCC High edition)

The runbook is validated with a **tabletop / simulated incident** (a controlled "game day") on the **GCC High** tenant, not by waiting for a real P1. Each test below has an explicit **PASS/FAIL gate**. The overall acceptance gate is: **a simulated incident is triaged correctly, comms are sent on cadence, diagnosis reaches a real root cause using the `*.us` consoles, an RCA is produced, and a permanent fix is logged.** Run the full sequence at least once before relying on the runbook, and re-run after any major change to the estate, the team, or **Microsoft's GCC High feature parity** (the gaps close over time).

> *Run readiness checks (T1–T2) with `scripts\INCIDENTG-Validation.ps1` (it connects with `-Environment USGov`) and capture the simulation evidence in a real **Incident Record Template – Incident – Lifecycle** flagged `SIMULATION`.*

T1. Responder readiness — roles & access (sovereign consoles) *(gate)*

• **Action:** Run `scripts\INCIDENTG-Validation.ps1`. Independently spot-check that a responder can open: **M365 admin (`portal.office365.us`)** → **Health** → **Service health**, **Entra (`entra.microsoft.us`)** → **Monitoring** → **Sign-in logs**, **Exchange admin (`admin.exchange.office365.us`)** → **Mail flow** → **Message trace**, **Azure (`portal.azure.us`)** → **Service Health** and a resource's **Resource health**, and **Defender (`security.microsoft.us`)** → **Incidents** (R3–R7).

• **PASS:** Validation returns **PASS** on roles, log access, and the `*.us` consoles all load for the responder account.

• **FAIL:** Any console denies access, a `*.com` URL was used by habit, or Validation reports a missing role → fix via the cited Requirement before continuing.

T2. Support-plan & escalation readiness (GCC High) *(gate)*

• **Action:** From the **Microsoft Escalation Sheet – Incident – Escalate**, confirm the recorded Microsoft entitlement matches what the portals show. Begin (do **not** submit) a support request in **Azure (**`portal.azure.us`**)** → **Help + support** → **Create a support request** and in **M365 (**`portal.office365.us`**)** → **Support** → **Help & support**; observe the **severity** options offered. Confirm the sheet does **not** list Threat Experts as an escalation path.

• **PASS:** Entitlement is documented against the GCC High support surface and the severity options the runbook claims are the ones actually offered (plan-dependent reality matches the sheet); no unavailable capability (Threat Experts) is listed as a lifeline.

• **FAIL:** The sheet over-states available severity, references a Commercial support path, cites Threat Experts, or the entitlement is unknown → correct the sheet / acquire the plan (R8).

T3. Triage correctness *(gate)*

• **Action:** Present 3 scripted scenarios (e.g. "all users can't sign in"; "one mailbox delayed"; "Teams calls failing for one site"). The responder classifies each with the **Severity & Impact Matrix – Incident – Triage** and names the **Incident Lead** (and, for P1/P2, Comms Lead + Scribe).

• **PASS:** All three are classified to the expected severity with correct response/cadence targets, and roles are named before diagnosis.

• **FAIL:** A scenario is mis-prioritized or no Incident Lead is named → re-train on A1/A3; refine the matrix wording.

T4. Service Health discrimination (sovereign) *(gate)*

• **Action:** In the simulation, require the responder to run the **Microsoft Service Health Playbook – Diagnose – Platform** *before* deep diagnosis and state whether the cause is Microsoft-side or tenant-side — using the `*.us` consoles.

• **PASS:** The responder checks both **M365 (**`portal.office365.us`**) Health** → **Service health** and **Azure (**`portal.azure.us`**)** → **Service Health** and correctly states is-this-Microsoft-or-us before diving in.

• **FAIL:** The responder skips Service Health, or checks the Commercial consoles, → enforce A6.1-first (in `*.us`) in Phase 2.

T5. Communication on cadence *(gate)*

• **Action:** During the simulation, the Communications Lead sends the **Initial** notification within the severity's response target and an **Update** at each cadence interval, using **Client Comms Templates – Incident – Communicate** (to an internal test recipient).

• **PASS:** Initial sent within target; at least two cadence Updates sent on time; tone is factual and jargon-free.

• **FAIL:** Late/missing comms or improvised wording → fix the cadence reminder and template usage (A4, R11).

T6. Diagnosis reaches root cause (no waiting on absent features) *(gate)*

• **Action:** Seed a diagnosable fault scenario (or replay a past incident). The responder follows the matching **Diagnostic Playbook – Incident – Diagnose** in the `*.us` consoles and produces a stated **root cause** with supporting evidence (trace ID, sign-in error code, Resource Health statement).

• **PASS:** A genuine root cause (the *why*) is reached with cited evidence — reached **manually**, without stalling to wait for Automated investigation (AIR) or a Threat Experts note that GCC High may not provide.

• **FAIL:** Diagnosis stops at the symptom, or the responder waits on an AIR/Threat-Experts capability that isn't available in GCC High → review the relevant A6 playbook and `gcc-high-parity-notes.md`; confirm log access (R4–R7).

T7. Remediation safety (US Gov endpoints) *(gate)*

• **Action:** Run `scripts\INCIDENTG-Remediation.ps1` against a named non-production object **without** `-Apply` (it connects with `-Environment USGov`); then with `-Apply` in a sanctioned test.

• **PASS:** Default run makes **no change** and prints the intended action (`-WhatIf`); `-Apply` performs only the named, scoped action against the **US Gov** endpoint and it is logged in the Incident Record (R12).

• **FAIL:** The script changes anything without `-Apply`, acts beyond the named object, or connects to a Commercial endpoint → stop; correct before live use.

T8. RCA produced *(gate)*

• **Action:** After the simulated resolution, complete the **RCA Template – Incident – Review** from the Incident Record timeline, including a 5-Whys root cause and corrective actions with owners/dates.

• **PASS:** A complete RCA exists within the agreed window with a real root cause and owned corrective actions.

• **FAIL:** RCA missing, late, or root-cause-free → enforce Phase 7; refine the template (A7).

T9. Permanent fix logged *(gate — overall)*

• **Action:** For at least one corrective action, open a **Problem Record – Incident – Permanent Fix** with an owner and due date and link it to the RCA and incident. Run `scripts\INCIDENTG-Report.ps1` and confirm MTTA/MTTR computed.

• **PASS:** A Problem Record exists for the permanent control, the loop is closed (incident → RCA → permanent fix), and the metrics report generates. (Where a Secure Score / MDVM item would be the natural control but is absent in GCC High, the control is tracked directly.)

• **FAIL:** No permanent fix tracked (the incident can recur) or metrics don't compute → the overall gate fails; do not certify the runbook until closed.

Acceptance gate (overall)

The runbook **passes** when, in a single end-to-end simulation on the GCC High tenant, **all of T1–T9 PASS**: readiness confirmed in the `*.us` consoles, triage correct, Service Health checked, comms on cadence,

diagnosis reaching root cause manually, safe remediation against US Gov endpoints, a produced RCA, and a logged permanent fix — with MTTA/MTTR captured. Record the simulation Incident Record and the Validation/Report CSVs as the certification evidence.

6. Troubleshooting — M365 & Azure Incident Response & Escalation Runbook (Government / GCC High edition)

Failure modes you hit while *running* the runbook (not the incidents themselves), grouped by area, each with a likely cause and a fix. For the incidents themselves, use the per-workload **Diagnostic Playbooks – Incident – Diagnose** in §03 (A6). The **first** section here is GCC-High-specific — it is where most sovereign-cloud responders lose time.

Sovereign-cloud connectivity (GCC High — read this first)

Symptom: `Connect-MgGraph` **fails, or signs you into the wrong tenant / returns "no data."**

- *Cause:* the module defaulted to the Commercial endpoint (`login.microsoftonline.com` / `graph.microsoft.com`). *Fix:* connect with `Connect-MgGraph -Environment USGov` (service root `graph.microsoft.us`). Consider `$PSDefaultParameterValues['Connect-MgGraph:Environment'] = 'USGov'` on responder workstations. (Source: [Microsoft Graph national cloud deployments](https://learn.microsoft.com/en-us/graph/deployments).)

Symptom: `Connect-AzAccount` **shows no Azure Government subscriptions.**

- *Cause:* connected to Azure Commercial. *Fix:* `Connect-AzAccount -Environment AzureUSGovernment`; work in `portal.azure.us`, not `portal.azure.com`.

Symptom: `Connect-ExchangeOnline` **fails or hits the wrong service.**

- *Cause:* missing US Gov environment name. *Fix:* `Connect-ExchangeOnline -ExchangeEnvironmentName O365USGovGCCHigh`. *(Verify the exact environment-name value for your ExchangeOnlineManagement module version.)*

Symptom: an admin console won't load or shows a different tenant.

- *Cause:* a `*.com` bookmark from Commercial habits. *Fix:* use the `*.us` consoles — `portal.office365.us`, `entra.microsoft.us`, `admin.exchange.office365.us`, `admin.gov.teams.microsoft.us`, `intune.microsoft.us`, `security.microsoft.us`, `portal.azure.us`. Full cited map in `gcc-high-parity-notes.md`.

Symptom: you're waiting for an automated investigation (AIR) or a Threat Experts note that never appears.

- *Cause:* those capabilities lag or are **not available** in GCC High. *Fix:* don't stall — **Threat Experts is not available** in GCC High and **AIR lags (verify)**; continue the **manual** A6 playbook. Advanced hunting (KQL) in `security.microsoft.us` is available for GA features. (Source: [Defender XDR for US Government](https://learn.microsoft.com/en-us/defender-xdr/usgov).)

Symptom: a Permanent-Fix action expected to be a Secure Score / MDVM recommendation isn't present.

• *Cause:* some Secure Score recommendations and MDVM functions (report inaccuracy, request CVE support) are **absent in GCC High**. *Fix:* track the control directly in the Problem Record; raise CVE/report-inaccuracy needs via a support case. (Source: [Defender for Endpoint for US Government](https://learn.microsoft.com/en-us/defender-endpoint/gov).)

Access & visibility (Detect / Diagnose)

Symptom: a responder can't open Service health / sign-in logs / message trace.

• *Cause:* missing reader/admin role (R3–R7). *Fix:* assign **Global Reader** / **Service Support Administrator** (Service health), **Reports/Security Reader** (sign-in logs), **Exchange Administrator** or custom RBAC (message trace), **Monitoring Reader** (Azure). Re-run `INCIDENTG-Validation.ps1`. Confirm you tested the `*.us` console, not the Commercial one.

Symptom: `Connect-MgGraph` **or** `Connect-ExchangeOnline` **connects but the account lacks consent/role.**

• *Cause:* module installed but the account lacks consent/role. *Fix:* `Install-Module ExchangeOnlineManagement, Microsoft.Graph -Scope CurrentUser`; confirm the role (R5/R4); for Graph, ensure the requested scopes are consented in the **GCC High** tenant.

Symptom: Identity Protection / Risky sign-ins blade is empty or hidden.

• *Cause:* **Entra ID P2** not licensed in the GCC High stack, or risk-detection behavior differs (R4). *Fix:* confirm the P2 entitlement (G5 Gov / EMS E5 for GCC High), **verify Identity Protection availability for GCC High**, or document that risk-based diagnosis is unavailable and rely on raw sign-in logs.

Symptom: Azure Monitor has no historical data at incident time.

• *Cause:* diagnostic settings weren't sending logs to Log Analytics before the incident (R7/G4). *Fix:* enable diagnostic settings now with adequate retention in Azure Government; you cannot retroactively create history.

Service Health discrimination (Triage)

Symptom: hours spent diagnosing a tenant fault that was actually a Microsoft outage.

• *Cause:* skipped the **Microsoft Service Health Playbook – Diagnose – Platform** (A6.1). *Fix:* enforce Service-Health-first in Phase 2; check **both** `portal.office365.us` → `Health` → `Service health` **and** `portal.azure.us` → `Service Health`.

Symptom: Service health shows nothing but users are clearly impacted.

• *Cause:* localized/region issue (US Gov region) not yet posted, or a tenant-specific cause. *Fix:* don't wait for Microsoft to post — proceed to the workload playbook; capture correlation IDs to escalate if it proves Microsoft-side.

Triage & roles

Symptom: incidents get mis-prioritized or response is inconsistent.

• *Cause:* matrix targets not aligned to SLAs, or no Incident Lead named (R11/A3). *Fix:* sync the **Severity & Impact Matrix – Incident – Triage** to contracts; require an Incident Lead before diagnosis.

Symptom: engineers swamped with status requests, can't focus on the fix.

- *Cause:* no Communications Lead routing comms. *Fix:* for P1/P2, always name a Communications Lead (A3); route all status questions through them.

Communication

Symptom: client complains they were "left in the dark."

- *Cause:* missed the cadence or never sent the Initial notification. *Fix:* set the cadence reminder at triage; send the Initial even with no answers yet; send "no change" updates on schedule (A4, R11).

Symptom: client confused or alarmed by an update.

- *Cause:* internal jargon or premature speculation on cause. *Fix:* use the templates verbatim; state impact, action, workaround, and next-update time only; keep cause out until evidenced.

Escalation to Microsoft

Symptom: can't set the severity you expected when opening a case.

- *Cause:* the severity is **plan-dependent** (R8) — e.g. Azure **Sev A** needs a paid Azure support plan. *Fix:* correct the **Microsoft Escalation Sheet – Incident – Escalate** to reflect what your GCC High plan actually permits; if you need faster response, acquire **Microsoft Unified** / an Azure support plan.

Symptom: you went looking for "Ask a Defender expert" (Threat Experts) and it isn't there.

- *Cause:* **Threat Experts is not available in GCC High.** *Fix:* the escalation path is a Microsoft **support case** in the `*.us` portal plus your internal chain — remove Threat Experts from the escalation sheet. (Source: [Defender XDR for US Government](https://learn.microsoft.com/en-us/defender-xdr/usgov).)

Symptom: Microsoft case stalls or bounces back for "more info."

- *Cause:* opened with symptoms only. *Fix:* attach the Phase-4 evidence (correlation IDs, message traces, sign-in error codes, Resource Health statements); reference the affected resource/region (US Gov region) precisely.

Remediation

Symptom: `INCIDENTG-Remediation.ps1` **didn't change anything.**

- *Cause:* it defaults to `-WhatIf` (by design). *Fix:* review the preview, then re-run with `-Apply`. Never add `-Apply` blindly.

Symptom: the remediation script connects but acts on the wrong tenant / fails auth.

- *Cause:* it reached a Commercial endpoint. *Fix:* the script connects with `-Environment USGov` by design — confirm you didn't override it; verify you signed in with the GCC High account at `login.microsoftonline.us`.

Symptom: a remediation made things worse / hit the wrong object.

- *Cause:* ran broadly instead of on a named object, or skipped the emergency-change rule (R12). *Fix:* always target a named object; follow the back-out in the Incident Record; record the change (a

compliance artifact in GCC High).

RCA → Permanent Fix loop

Symptom: the same incident keeps recurring.

• *Cause:* fixes stopped at mitigation; no permanent fix tracked. *Fix:* enforce Phase 7 — complete the **RCA Template – Incident – Review** with 5-Whys and open a **Problem Record – Incident – Permanent Fix** with an owner and date.

Symptom: RCA is late, thin, or missing.

• *Cause:* no contemporaneous timeline (Scribe role skipped) or no agreed window. *Fix:* keep the Scribe's timeline live during the incident; set an RCA SLA (e.g. ≤5 business days for P1) and gate closure on the RCA existing.

Symptom: MTTA/MTTR can't be computed.

• *Cause:* missing Detected/Acknowledged/Resolved timestamps in the Incident Record. *Fix:* make those fields mandatory in the **Incident Record Template – Incident – Lifecycle** (A2); re-run `INCIDENTG-Report.ps1`.

7. Best Practices — M365 & Azure Incident Response & Escalation Runbook (Government / GCC High edition)

These are the operating disciplines that make the runbook effective in practice on a **GCC High** tenant. They assume the named artifacts from §03 are in place and reused verbatim.

Sovereign-cloud discipline (GCC High-specific)

• **Every URL is a `*.us` URL.** Bookmark the sovereign consoles (`portal.office365.us`, `entra.microsoft.us`, `admin.exchange.office365.us`, `admin.gov.teams.microsoft.us`, `intune.microsoft.us`, `security.microsoft.us`, `portal.azure.us`) and use them. A `*.com` habit sends you to a Commercial tenant that doesn't hold your client's data. Keep `gcc-high-parity-notes.md` one click away.

• **Always pass the US Gov environment in PowerShell.** `Connect-MgGraph -Environment USGov`, `Connect-AzAccount -Environment AzureUSGovernment`, `Connect-ExchangeOnline -ExchangeEnvironmentName O365USGovGCCHigh`. The most common "it won't connect" error mid-incident is a missing `-Environment USGov`. Consider setting `$PSDefaultParameterValues['Connect-MgGraph:Environment'] = 'USGov'` on responder workstations.

• **Know the parity gaps cold.** Do not build a step around a capability GCC High lacks: **no Threat Experts**, **AIR lags (verify)**, **no enterprise-IoT Defender**, **some Secure Score / MDVM items absent**. Re-verify these each quarter — Microsoft closes gaps, and a stale "not available" note is as harmful as a wrong URL.

• **Stay in-boundary.** Keep evidence, the war-room, and the system of record within approved in-scope services; the whole point of GCC High is that data doesn't leak to Commercial. The audit trail is part of the compliance posture, not overhead.

Triage & prioritization

• **Service Health first, always — in `*.us`.** Run the **Microsoft Service Health Playbook – Diagnose – Platform** before any deep local diagnosis, in `portal.office365.us` and `portal.azure.us`. The cheapest minute in any incident is the one that tells you it's a Microsoft outage.

• **Prioritize by impact × risk × effort.** Take the highest-impact, lowest-risk, lowest-effort mitigation first. A 30-second workaround that restores 200 users beats a 2-hour "proper" fix while they wait.

• **Mitigate before you perfect.** Restore service first; do the elegant root-cause fix after. Record the mitigation and the resolution as separate timeline entries (they drive different metrics).

• **Sever the symptom from the cause.** "It works again" after a restart is a mitigation, not a root cause. Phase 7 is non-negotiable for P1/P2.

Roles & coordination

• **Name the Incident Lead before diagnosis.** Even a one-person response names the Lead in the record. Diffuse ownership is how incidents drift.

• **Protect the engineers.** For P1/P2, route all client/stakeholder status requests through the **Communications Lead** so the people fixing the problem can fix the problem.

• **The Scribe is not optional under pressure.** A contemporaneous, timestamped timeline is what makes the RCA accurate and fast — and, in a regulated estate, what satisfies the audit.

Client communication

• **Communicate early, even with little to say.** The Initial notification builds trust precisely because it goes out before you have answers. Silence is what erodes confidence.

• **Hold the cadence religiously.** Send the Update at every interval, including "still investigating, next update by HH:MM." Missing a promised update is its own incident.

• **Calm, factual, jargon-free.** Clients want impact, what you're doing, any workaround, and the next update time — not packet captures.

• **Never speculate publicly on cause** until you have evidence; preliminary cause goes in Resolution, confirmed cause goes in the Post-Incident summary.

Diagnosis

• **Capture correlation IDs as you go** (message-trace IDs, sign-in request IDs, Resource Health correlation IDs). They are gold for a Microsoft escalation and worthless if you have to re-derive them.

• **Follow the named playbook in order.** The ordering encodes the fastest path to root cause for that workload; ad-hoc clicking is slower and skips checks.

• **Don't wait on features GCC High may not have.** If you expected AIR to have auto-investigated or a Threat Experts insight to appear and it hasn't, that's expected here — continue the **manual** playbook rather than stalling. Advanced hunting (KQL) in `security.microsoft.us` is available for GA features and is a strong manual tool.

• **Mind the dependency chain.** Teams leans on Exchange (calendar) and SharePoint (files); SharePoint/OneDrive lean on identity. Diagnose the dependency when the surface symptom doesn't add up.

Escalation to Microsoft

• **Know your GCC High plan before the incident.** Re-confirm the **Microsoft Escalation Sheet – Incident – Escalate** entitlement periodically — *severity options and response are plan-dependent* and confirmed in the `*.us` support surface. Azure **Sev A** generally needs a paid Azure support plan; **Microsoft Unified** gives managed-service breadth.

• **The escalation is a support case — there is no Threat Experts lifeline in GCC High.** Open the case in the GCC High portal and attach the Phase-4 evidence; a case with correlation IDs and message traces moves far faster.

• **Escalate internally on a clock, too.** Define when the Incident Lead pulls in a senior engineer or management — don't let a stuck P1 stay quietly stuck.

Remediation safety

• `-WhatIf` **by default,** `-Apply` **deliberately.** `INCIDENTG-Remediation.ps1` previews by default for a reason, and connects only to the **US Gov** endpoint. Read the preview; apply only the named, scoped action.

• **Act on named objects only.** Revoke *this* user's sessions; re-enable *this* connector. Never run a remediation broadly under incident pressure.

• **Emergency change still has a record and a back-out.** Define what may be done before approval and what may not (R12); log every change in the Incident Record — the change trail is a compliance artifact in GCC High.

RCA → Permanent Fix loop

• **5-Whys to the real cause.** Stop at "the certificate expired" and you'll be back; ask why it expired un-noticed and you'll add the monitor that prevents recurrence.

• **Every RCA action has an owner and a date.** Actions without owners don't happen.

• **Convert fixes into controls.** A permanent fix is usually a new alert/monitor, a config hardening, a policy, or a runbook update — track it as a **Problem Record – Incident – Permanent Fix** to closure. Where the natural control would be a Secure Score / Defender Vulnerability Management recommendation that is **absent in GCC High**, track the control directly instead of citing a feature you don't have.

• **Feed the metrics.** Run `INCIDENTG-Report.ps1` after each incident; a falling MTTA/MTTR and falling repeat-incident count is the proof the program works.

Program hygiene

• **Run a tabletop quarterly — and re-check parity while you do.** The §05 simulation keeps the team fluent and surfaces access gaps before a real P1 does; use the same cadence to re-verify the GCC High parity notes against Microsoft's current docs.

• **Keep access least-privilege and PIM-eligible.** Standing Global Admin for the whole team is an audit finding and a breach blast-radius; reader-first with eligible scoped admin is the target.

• **Keep the matrix and templates in sync with contracts.** When a client SLA changes, update the **Severity & Impact Matrix – Incident – Triage** the same day.

8. Deliverables Summary — M365 & Azure Incident Response & Escalation Runbook (Government / GCC High edition)

This package is a complete, enterprise-grade incident & problem management kit for leading high-priority and escalated incidents across **Microsoft 365 GCC High and Azure Government**. It contains the documentation runbook (with an explicit, cited Commercial-vs-GCC-High parity map), five PowerShell tools that connect to US Gov endpoints, and a hero image.

Documentation

File	What it covers
`01-solution-overview.md`	What the runbook is, the **Detect** → **Triage** → **Communicate** → **Diagnose** → **Resolve** → **RCA** → **Permanent Fix** phase model, the named artifacts, the per-workload diagnostic playbooks, benefits, and honest limitations — plus a **"Commercial vs GCC High — what's different"** section (portals, endpoints, feature parity).
`02-requirements.md`	R1–R12 prerequisites (Description / Why / Verify / Implement-if-missing) with every *Verify* step in the `*.us` consoles / US Gov PowerShell: responder roles + reader access, Service Health access, Entra/Exchange/Azure log access, a **GCC High** support plan, a PSA, comms channels, SLAs, and change control.
`global-requirements-managed.md`	Baseline requirements common to every Managed-category engagement (tenant/subscriptions, roles, licensing, monitoring + system of record, change control, backup/rollback).
`03-implementation.md`	Part A stands up the matrix, templates, playbooks, escalation sheet, RCA, and responder access; Part B is the per-incident runbook with full **GCC High (`*.us`)** console navigation paths and inline sovereign-cloud caveats.

File	What it covers
`gcc-high-parity-notes.md`	The cited **Commercial-vs-GCC-High reference**: admin portal URLs, PowerShell/API endpoints, feature parity gaps (Threat Experts, AIR, Defender for IoT, Secure Score / MDVM items, advanced hunting, Sentinel, Identity Protection), and compliance framing — each with a Microsoft source.
`05-testing.md`	Numbered tests (T1–T9) with PASS/FAIL gates, validated through a tabletop/simulated incident on the GCC High tenant: readiness in `*.us` consoles, triage correct, comms on cadence, diagnosis reaching root cause manually, RCA produced, permanent fix logged.
`06-troubleshooting.md`	Common failure modes across sovereign-cloud connectivity, access/visibility, Service Health discrimination, per-workload diagnosis, escalation, remediation, comms, and the RCA loop — with fixes.
`07-best-practices.md`	The operating disciplines: sovereign-cloud discipline (`*.us` + `-Environment USGov` + parity awareness), Service-Health-first triage, role coordination, client comms, evidence-led diagnosis and escalation, remediation safety, and the RCA→permanent-fix loop.
`executive-summary.md`	One-page business case and outcomes for sponsors/leadership.

PowerShell scripts

All scripts carry a `.SYNOPSIS` header with PURPOSE / HOW TO RUN / EXPECTED RESULTS and a NEUTRAL white-label `.NOTES`, and connect to **Microsoft Graph US Gov** (`-Environment USGov`) / **Azure Government** (`-Environment AzureUSGovernment`) endpoints. Read-only and discovery scripts make no changes; the remediation and scope-targeting scripts default to `-WhatIf` and require `-Apply` to act, on **named objects only**.

Script	What it does
`scripts\INCIDENTG-Discovery.ps1`	**Live incident snapshot (read-only, US Gov).** Pulls current Microsoft 365 + Azure **Service Health**, recent **high-severity sign-ins / risk detections** from Entra, and an **Azure Government Resource Health** snapshot into one CSV to seed diagnosis — via `graph.microsoft.us` and Azure Government.

Script	What it does
`scripts\INCIDENTG-Validation.ps1`	**Responder readiness (PASS/FAIL, US Gov).** Verifies responders hold the required roles, can read the key logs (Service Health, sign-in, message trace, Azure Monitor), and that a **GCC High** Microsoft support plan is in place; emits PASS/FAIL per check for the §05 readiness gates.
`scripts\INCIDENTG-Report.ps1`	**Incident metrics report.** Reads incident timeline data and computes **MTTA/MTTR** and per-severity counts/trends to CSV — the program dashboard that proves the runbook is working. (Offline; no tenant access required.)
`scripts\INCIDENTG-Remediation.ps1`	**Safe common fixes (US Gov).** Performs scoped, well-understood remediations for a **named object** (e.g. revoke a user's sessions, re-enable a disabled mail-flow connector) against the US Gov endpoints. Runs read-only / `-WhatIf` by default; requires `-Apply` to change anything.
`scripts\INCIDENTG-Set-ScopeTargeting.ps1`	**Responder access + scope setup (US Gov).** Creates the `Responder Access Group – Identity – Response` Entra group and assigns the reader/scoped-admin roles responders need; supports `-WhatIf` preview before `-Apply`. Connects with `-Environment USGov`.

Hero image

File	Purpose
`INCIDENTG-hero.png`	Listing/marketing hero image for the solution.

> ***Naming convention reminder:*** *the scripts and runbook share the object-naming standard* `<Purpose> – <Scope> – <Stage>` *(e.g.* `Responder Access Group – Identity – Response`, `Microsoft Escalation Sheet – Incident – Escalate`, `GCC High Parity Notes – Incident – Sovereign`*) so outputs cross-reference cleanly across discovery, validation, remediation, and reporting.*
>
> ***Sovereign-cloud reminder:*** *every console path in the documentation uses the* `*.us` *URL and every script connects with a* ***US Gov*** *environment flag. The* `gcc-high-parity-notes.md` *file is the single cited source of truth for those URLs, endpoints, and feature gaps — re-verify it against Microsoft's current docs each quarter, as parity gaps close over time.*

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

INCIDENTG-Discovery.ps1

Purpose: Seeds the Diagnose phase of an incident on a GCC High tenant by pulling, in one pass: current Microsoft 365 Service Health issues and Azure Government Service Health events; recent failed / high-risk sign-ins (and, if Entra ID P2 is licensed, risk detections); and an Azure Government Resource Health snapshot. Output is one CSV for the Incident Record. No changes are made — strictly read-only against the US Gov endpoints (graph.microsoft.us, Azure Government). HOW TO RUN powershell -ExecutionPolicy Bypass -File .\INCIDENTG-Discovery.ps1 First run installs modules: Install-Module Microsoft.Graph, Az -Scope CurrentUser Graph scopes requested: ServiceHealth.Read.All, AuditLog.Read.All, IdentityRiskEvent.Read.All Connects with -Environment USGov (Graph) and -Environment AzureUSGovernment (Az); sign in with a Global Reader / Security Reader / Monitoring Reader account at login.microsoftonline.us. EXPECTED RESULTS A CSV 'Incident Snapshot - Estate - Detect.csv' with one row per finding (Source, Category, Severity, Name, Status, Detail, Observed) plus a console summary by source. Read-only: performs no changes. Pass = the CSV is produced with the expected finding rows. .NOTES White-label — the repeatable way. Read-only against Microsoft Graph US Gov endpoints. #>

How to run: powershell -ExecutionPolicy Bypass -File .\INCIDENTG-Discovery.ps1 First run installs modules: Install-Module Microsoft.Graph, Az -Scope CurrentUser Graph scopes requested: ServiceHealth.Read.All, AuditLog.Read.All, IdentityRiskEvent.Read.All Connects with -Environment USGov (Graph) and -Environment AzureUSGovernment (Az); sign in with a Global Reader / Security Reader / Monitoring Reader account at login.microsoftonline.us. EXPECTED RESULTS A CSV 'Incident Snapshot - Estate - Detect.csv' with one row per finding (Source, Category, Severity, Name, Status, Detail, Observed) plus a console summary by source. Read-only: performs no changes. Pass = the CSV is produced with the expected finding rows. .NOTES White-label — the repeatable way. Read-only against Microsoft Graph US Gov endpoints. #>

INCIDENTG-Remediation.ps1

Purpose: Performs a small set of well-understood, scoped remediations during an incident on a GCC High tenant, always against a single NAMED object — never broadly: RevokeSessions (revoke a compromised user's refresh tokens), EnableConnector (re-enable a named Exchange Online mail-flow connector), and EnableInboundRule (re-enable a named transport rule). It connects only to the US Gov endpoints and is preview-only by default: it changes NOTHING unless you pass -Apply, and only ever touches the object you name. HOW TO RUN Preview (default, makes NO change): powershell -ExecutionPolicy Bypass -File .\INCIDENTG-Remediation.ps1 -Action RevokeSessions -Target "user@agency.gov" Apply for real (deliberate): powershell -ExecutionPolicy Bypass -File .\INCIDENTG-Remediation.ps1 -Action EnableConnector -Target "Inbound from Gateway" -Apply First run installs modules: Install-Module Microsoft.Graph, ExchangeOnlineManagement -Scope CurrentUser Graph scopes requested (RevokeSessions): User.RevokeSessions.All, User.ReadWrite.All Connects with -Environment USGov (Graph) and -ExchangeEnvironmentName O365USGovGCCHigh (EXO); sign in at login.microsoftonline.us. EXPECTED RESULTS Without -Apply: prints the planned action and exits without changing anything (pass = no change). With -Apply: performs only the single named action against the US Gov endpoint and prints the result for the Incident Record. .NOTES White-label — the repeatable

way. Read-only against Microsoft Graph US Gov endpoints. #>

How to run: Preview (default, makes NO change): powershell -ExecutionPolicy Bypass -File .\INCIDENTG-Remediation.ps1 -Action RevokeSessions -Target "user@agency.gov" Apply for real (deliberate): powershell -ExecutionPolicy Bypass -File .\INCIDENTG-Remediation.ps1 -Action EnableConnector -Target "Inbound from Gateway" -Apply First run installs modules: Install-Module Microsoft.Graph, ExchangeOnlineManagement -Scope CurrentUser Graph scopes requested (RevokeSessions): User.RevokeSessions.All, User.ReadWrite.All Connects with -Environment USGov (Graph) and -ExchangeEnvironmentName O365USGovGCCHigh (EXO); sign in at login.microsoftonline.us. EXPECTED RESULTS Without -Apply: prints the planned action and exits without changing anything (pass = no change). With -Apply: performs only the single named action against the US Gov endpoint and prints the result for the Incident Record. .NOTES White-label — the repeatable way. Read-only against Microsoft Graph US Gov endpoints. #>

INCIDENTG-Report.ps1

Purpose: Reads an exported set of incident records (from the PSA / Incident Record Template) and computes the program metrics that prove the runbook is working on the GCC High estate: per-incident MTTA (Detected to Acknowledged) and MTTR (Detected to Resolved), plus per-severity counts and average MTTA/MTTR. Output is a metrics CSV plus a console dashboard. This script is fully offline — it only reads the input file and needs no tenant access or US Gov endpoint, so it never leaves the boundary. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\INCIDENTG-Report.ps1 -InputCsv ".\Incidents - Export - Lifecycle.csv" The input CSV needs at least these columns (ISO-8601 UTC timestamps; ResolvedUtc may be blank): IncidentId, Severity, DetectedUtc, AcknowledgedUtc, ResolvedUtc (No Graph scopes and no -Environment flag are required — this tool does not connect to any cloud.) EXPECTED RESULTS A CSV 'Incident Metrics - Program - Review.csv' with per-incident MTTA/MTTR (minutes) and a console summary of counts and average MTTA/MTTR by severity. Pass = the metrics CSV is written and the per-severity summary prints. .NOTES White-label — the repeatable way. Read-only against Microsoft Graph US Gov endpoints. #>

How to run: powershell -ExecutionPolicy Bypass -File .\INCIDENTG-Report.ps1 -InputCsv ".\Incidents - Export - Lifecycle.csv" The input CSV needs at least these columns (ISO-8601 UTC timestamps; ResolvedUtc may be blank): IncidentId, Severity, DetectedUtc, AcknowledgedUtc, ResolvedUtc (No Graph scopes and no -Environment flag are required — this tool does not connect to any cloud.) EXPECTED RESULTS A CSV 'Incident Metrics - Program - Review.csv' with per-incident MTTA/MTTR (minutes) and a console summary of counts and average MTTA/MTTR by severity. Pass = the metrics CSV is written and the per-severity summary prints. .NOTES White-label — the repeatable way. Read-only against Microsoft Graph US Gov endpoints. #>

INCIDENTG-Set-ScopeTargeting.ps1

Purpose: Stands up responder access and scope for the runbook on a GCC High tenant (see 03-implementation.md, A8): creates or reuses the role-assignable Entra security group 'Responder Access Group - Identity - Response', optionally adds named responder users, and assigns the least-privilege reader directory roles responders need for the Diagnose phase (Global Reader, Service Support Administrator, Reports Reader, Security Reader). It connects only to the US Gov Graph endpoint and previews by default — pass -Apply to make changes. It intentionally does NOT grant scoped admin (Exchange/Teams/SharePoint/ Intune Administrator), which should be PIM-eligible and granted deliberately per the requirements. HOW TO RUN Preview (default, makes NO change): powershell

-ExecutionPolicy Bypass -File .\INCIDENTG-Set-ScopeTargeting.ps1 -Members "a@agency.gov","b@agency.gov" Apply for real: powershell -ExecutionPolicy Bypass -File .\INCIDENTG-Set-ScopeTargeting.ps1 -Members "a@agency.gov","b@agency.gov" -Apply First run installs the module: Install-Module Microsoft.Graph -Scope CurrentUser Graph scopes requested: Group.ReadWrite.All, RoleManagement.ReadWrite.Directory, User.Read.All Connects with -Environment USGov; run as a Privileged Role Administrator / Global Administrator and sign in at login.microsoftonline.us. EXPECTED RESULTS Without -Apply: prints the group + role assignments it WOULD make (pass = no change). With -Apply: creates the group, adds members, and assigns the reader roles; prints a summary. .NOTES White-label — the repeatable way. Read-only against Microsoft Graph US Gov endpoints. #>

How to run: Preview (default, makes NO change): powershell -ExecutionPolicy Bypass -File .\INCIDENTG-Set-ScopeTargeting.ps1 -Members "a@agency.gov","b@agency.gov" Apply for real: powershell -ExecutionPolicy Bypass -File .\INCIDENTG-Set-ScopeTargeting.ps1 -Members "a@agency.gov","b@agency.gov" -Apply First run installs the module: Install-Module Microsoft.Graph -Scope CurrentUser Graph scopes requested: Group.ReadWrite.All, RoleManagement.ReadWrite.Directory, User.Read.All Connects with -Environment USGov; run as a Privileged Role Administrator / Global Administrator and sign in at login.microsoftonline.us. EXPECTED RESULTS Without -Apply: prints the group + role assignments it WOULD make (pass = no change). With -Apply: creates the group, adds members, and assigns the reader roles; prints a summary. .NOTES White-label — the repeatable way. Read-only against Microsoft Graph US Gov endpoints. #>

INCIDENTG-Validation.ps1

Purpose: Validates that the response team is ready to run the runbook on a Microsoft 365 GCC High tenant BEFORE a real incident: the responder access group exists, responders hold the required roles, the key logs are readable (Service Health, Entra sign-in logs, Exchange message trace, Azure Government subscription/Monitor), and a Microsoft support plan is reachable for escalation. Emits PASS/FAIL per check with a non-zero exit code on any FAIL. Read-only against US Gov endpoints. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\INCIDENTG-Validation.ps1 First run installs modules: Install-Module Microsoft.Graph, ExchangeOnlineManagement, Az -Scope CurrentUser Graph scopes requested: Directory.Read.All, RoleManagement.Read.Directory, ServiceHealth.Read.All, AuditLog.Read.All Connects with -Environment USGov (Graph), -ExchangeEnvironmentName O365USGovGCCHigh (EXO), and -Environment AzureUSGovernment (Az). Run as (or impersonating) a responder account; sign in at login.microsoftonline.us. EXPECTED RESULTS A console table of checks each marked PASS or FAIL, an exported results CSV, and exit code 1 if any check FAILS. Read-only. Pass = every readiness check returns PASS and exit code is 0. .NOTES White-label — the repeatable way. Read-only against Microsoft Graph US Gov endpoints. #>

How to run: powershell -ExecutionPolicy Bypass -File .\INCIDENTG-Validation.ps1 First run installs modules: Install-Module Microsoft.Graph, ExchangeOnlineManagement, Az -Scope CurrentUser Graph scopes requested: Directory.Read.All, RoleManagement.Read.Directory, ServiceHealth.Read.All, AuditLog.Read.All Connects with -Environment USGov (Graph), -ExchangeEnvironmentName O365USGovGCCHigh (EXO), and -Environment AzureUSGovernment (Az). Run as (or impersonating) a responder account; sign in at login.microsoftonline.us. EXPECTED RESULTS A console table of checks each marked PASS or FAIL, an exported results CSV, and exit code 1 if any check FAILS. Read-only. Pass = every readiness check returns PASS and exit code is 0. .NOTES White-label — the repeatable way. Read-only against Microsoft Graph US Gov endpoints. #>